

CHAPTER 4:

LOOPS

Overview

- Elements of Repetition Structure to cover
  1. Definition of Repetition Structure
  2. Type of Repetition Structure
  3. Draw flowchart related to Repetition Structure
  4. Write program with Repetition Structure

Overview

- REPETITION statements – allow a group of instructions to be executed repeatedly, until a logical condition has been satisfied.

- Variation of REPETITION statements
  - for statement
  - while statement
  - do-while statement
  - nested for statement

Loop/Iterative Statements

'while' statement

syntax:
while (<loop condition>)
<statements>

Java Code:

```
public class WhileLoopDemo {
        public static void main(String[] args) {
            int count = 1;
            System.out.println("Printing Numbers from 1 to 10");
            while (count <= 10) {
                    //System.out.println(count++);
                    System.out.println(count);
                    count++;
}
```

Flowchart “while’ structure

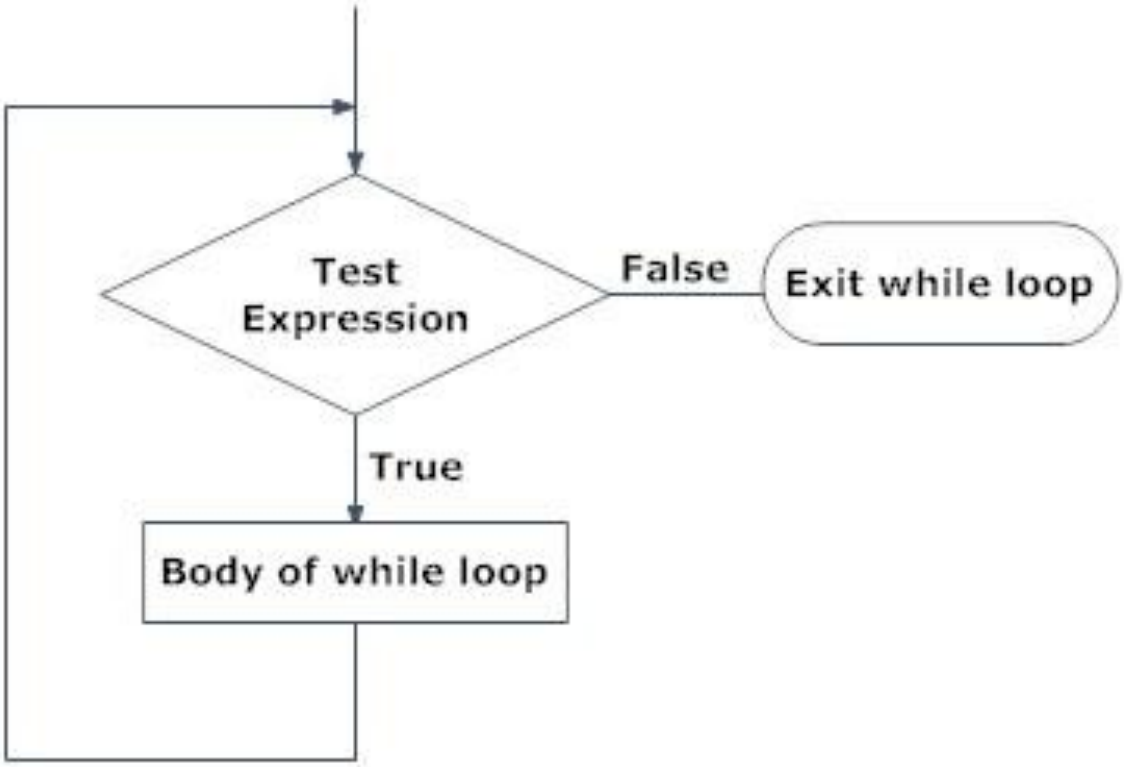
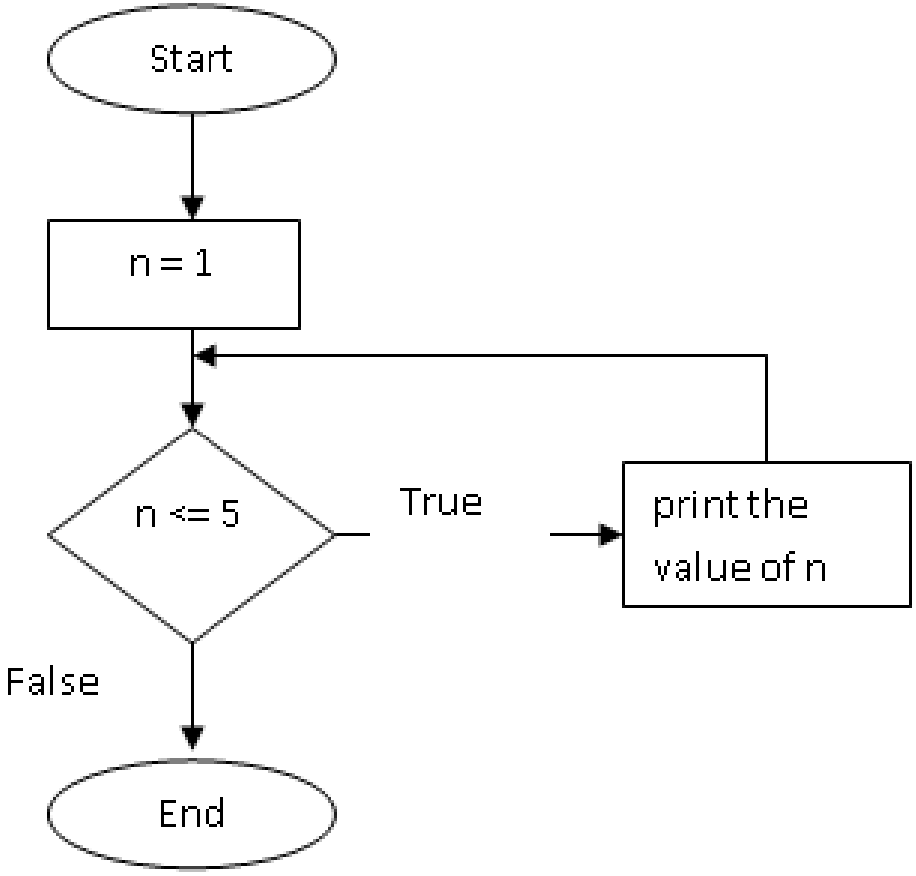


Figure: Flowchart of while loop

Example Flowchart “while’ structure



Loop/Iterative Statements

'do…while' statement

syntax:

```
do
<loop body>
while (<loop condition>);
```

Java Code:

```
public class DoWhileLoopDemo {
        public static void main(String[] args) {
            int count = 1;
            System.out.println("Printing Numbers from 1 to 10");
            do {
                System.out.println(count++); }
            while (count <= 10); } }
```

Flowchart

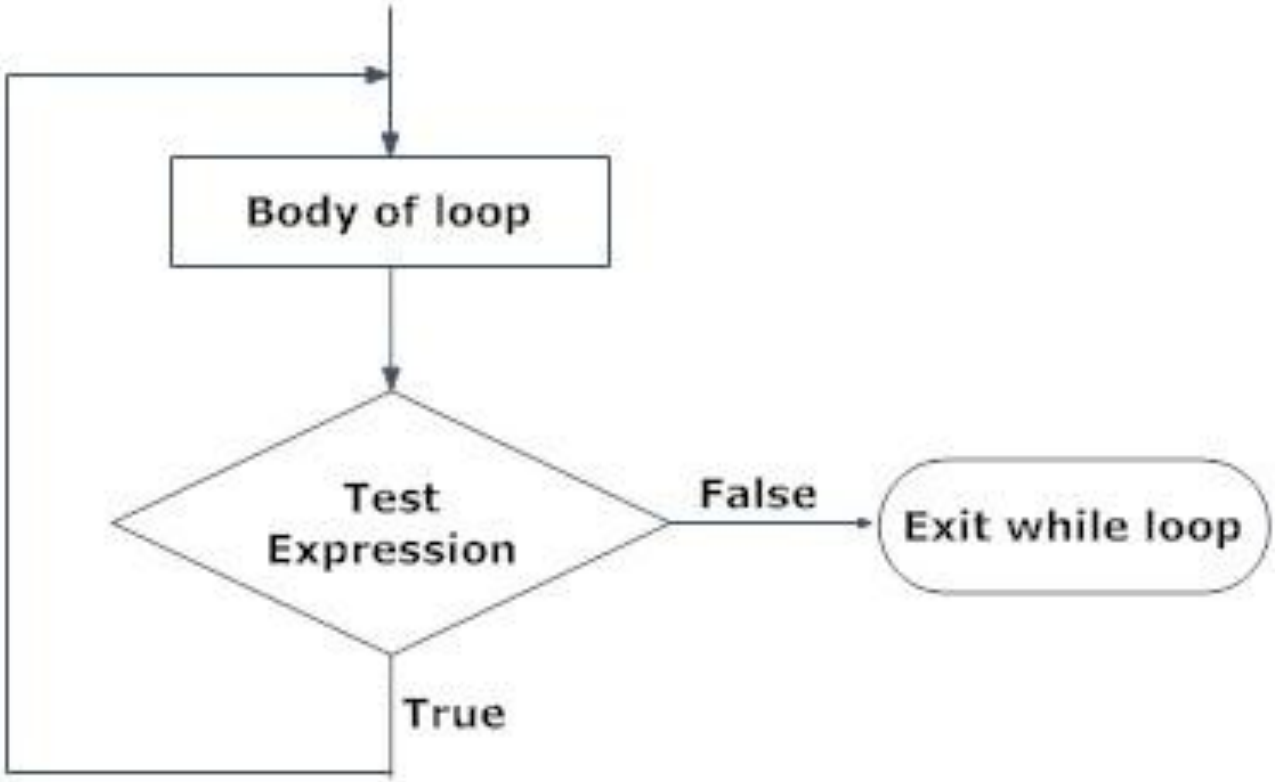
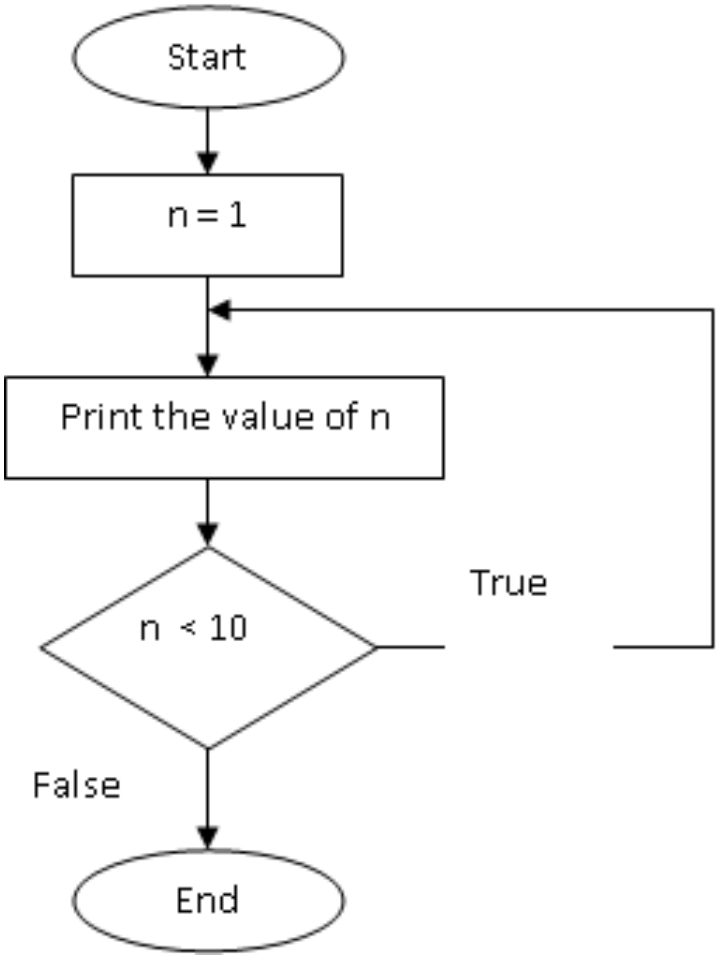


Figure: Flowchart of do...while loop

Example



Loop/Iterative Statements

'for' statement

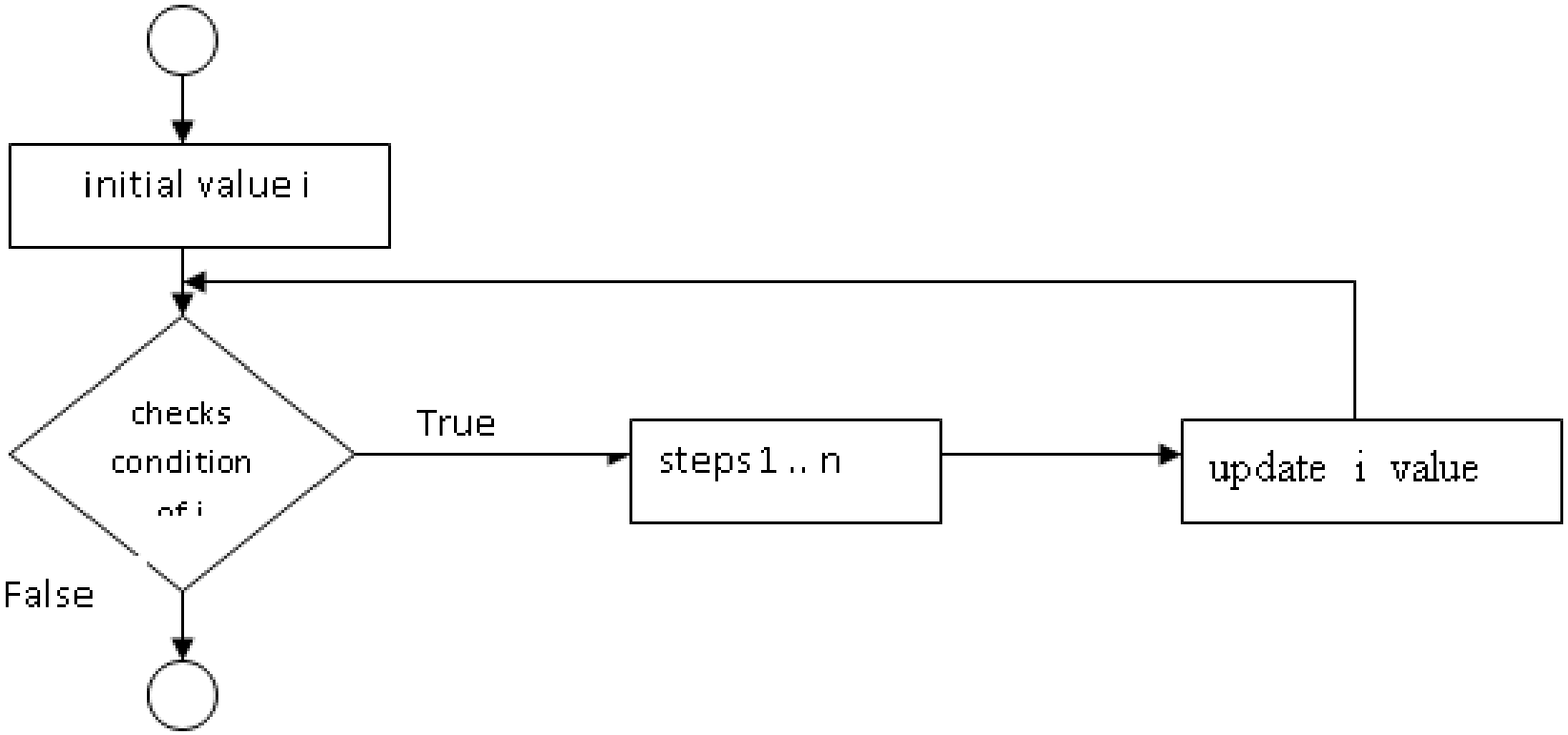
syntax:

for (<initialization>; <loop condition>; <increment expression>)
<loop body>

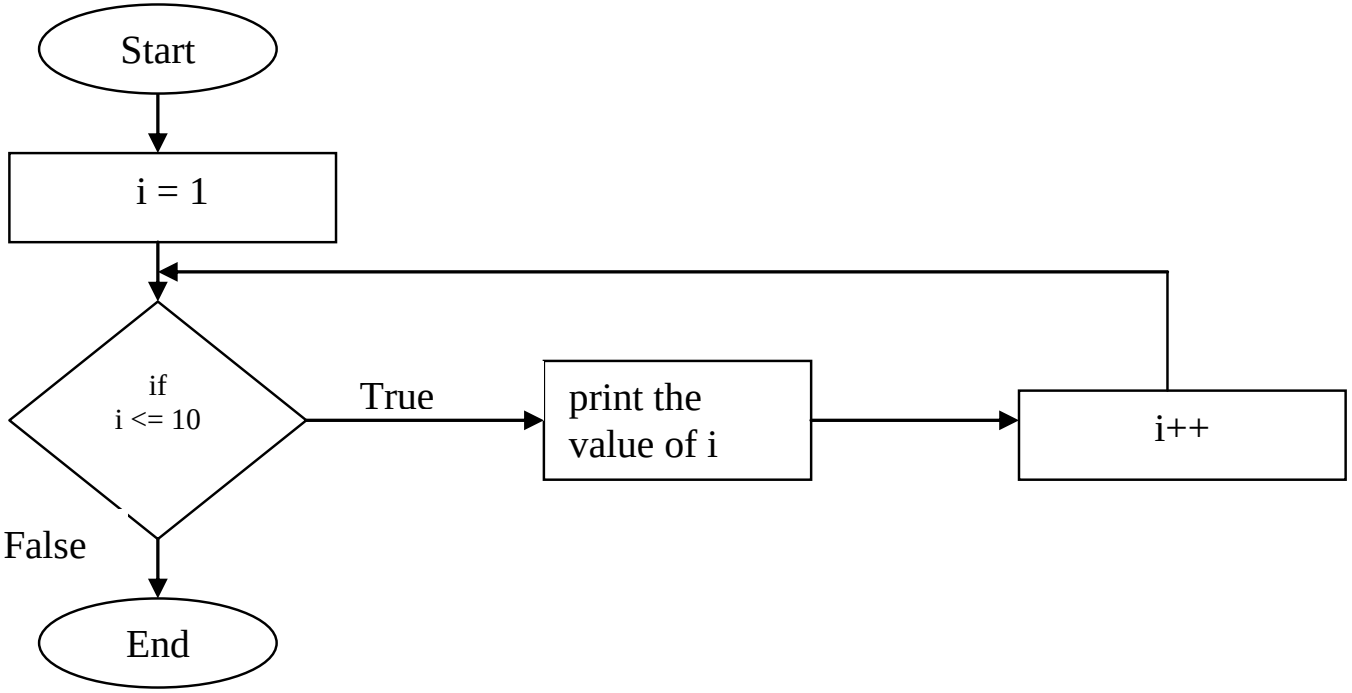
Java Code:

```
public class ForLoopDemo {
    public static void main(String[] args) {
            System.out.println("Printing Numbers from 1 to 10");
            for (int count = 1; count <= 10; count++) {
               System.out.println(count); } } }
```

‘for’ structure flowchart



Example



Loop/Iterative Statements

'for' statement represented with while structure

Syntax:

```
expression1
while ( expression2 ) {
        statement
        expression3;
}
```

Java Code:

```
public class ForLoopDemo {
    public static void main(String[] args) {
        System.out.println("Printing Numbers from 1 to 10");
        int count = 1;
        while (count <= 10 ) {
            System.out.println(count);
             count++ }
        } }
```

Nested Looping

When nested, the inner loop iterates from beginning to end for each single iteration of the outer loop

Java Code:

```
public class  ForLoop{
  public static void main(String[] args){
    for(int i = 1;i <= 5;i++){
      for(int j = 1;j <= i;j++){
        System.out.print(i);
      }
      System.out.println();  }
      }
}
```

Inner Loop

Outer Loop

Transfer Statements

'break' statement

syntax:
break; // the unlabeled form
break <label>; // the labeled form to terminate nested looping structure

Java Code:

```
public class BreakExample {
    public static void main(String[] args) {
        System.out.println("Numbers 1 - 10");
        for (int i = 1;i <=10; ++i) {
            if (i == 5)
                break; // break loop when i is 5
            System.out.println(i + "\t"); }
            System.out.println("Broke out of loop at i = "+i); } }
```

Transfer Statements

‘continue’ statement

syntax:

continue; // the unlabeled form

continue <label>; // the labeled form

Java Code:

```
public class ContinueExample {
    public static void main(String[] args) {
        System.out.println("Numbers 1 - 10");
        for (int i = 1;i<=10; ++i) {
            if (i == 5)
                continue; // Skip remaining code in loop when i is 5
            System.out.println(i + "\t"); }
            System.out.println(“Skip printing 5”); } }
```

Transfer Statements

‘**return’ statement :** The return statement exits from the current method, and control flow returns to where the method was invoked.

syntax:
return; // the unlabeled form
return <value/expression>; // the labeled form

Java Code:

```
public class ReturnExample {
    public static void main(String[] args) {
        int result;
        for (int i = 1;i <=10; ++i) {
            result = square(i);
            System.out.println(“The square of” +I “is” =result +”\n”); }}
     public int square(int y){
         return y * y; // return square of y
  }
```

Comparing String

Method 1

```
if (upwd.equals(pwd))
```

Method 2

```
If (choose.equals("Yes"))
```

Method 3

```
while(repeat2.compareTo("#")!=0)
```

Compare Character

If (choice.charAt(0)=='Y')